به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴) ۳ تیر ۹۸

دانشگاه صنعتی شریف مدرس: مهدی صفرنژاد

آزمون پایان‌ترم

زمان این امتحان ۱۵۰ دقیقه و مجموع نمرات آن ۱۰۰ است. سؤالات در چهار صفحه (دو برگهٔ پشت و رو) قرار دارند. توضیحات ابتدای سؤالات را به دقت بخوانید و پاسخ هر سؤال را در برگه‌های جداگانه بنویسید. پاسخ بخش‌های مختلف هر سؤال را نیز به ترتیب در برگهٔ پاسخ‌نامه بنویسید.

سؤال ۱. [۳۳ نمره] سؤال‌های جواب کوتاه. در سؤال‌های این قسمت معمولاً یک جواب کوتاه و/یا یک توضیح کوتاه حدوداً بین یک تا دو خط مدنظر است؛ مگر این که صورت آن قسمت به صراحت خواستهٔ دیگری داشته باشد.

آ. **[۴ نمره]** در درهم‌سازی کامل، زمان یک جست‌وجو حداکثر چقدر می‌تواند باشد؟ چرا؟

پاسخ: $O(1)$. هر جست‌وجو به دقیقاً دو بار محاسبهٔ تابع درهم‌سازی و دو بار نگاه به خانه‌های جدول و مقایسه نیاز دارد.

ب. **[۴ نمره]** اگر ضرب دو عدد را با الگوریتم فیور در زمان $\Theta(d \log d \cdot ۲^{O(\log^* d)})$ پیاده‌سازی کنیم که d تعداد رقم‌های عدد است، زمان تقسیم دو عدد با استفاده از روش نیوتون و الگوریتم فیور چقدر خواهد بود؟ چرا؟

پاسخ: $\Theta(d \log d \cdot ۲^{O(\log^* d)})$ در کلاس دیدیم که با این روش، زمان تقسیم با ضرب به صورت مجانبی برابر می‌شود.

پ. **[۴ نمره]** تبدیل باروز ویلر $baba چیست؟ نحوهٔ به دست آوردن پاسخ را به اختصار توضیح دهید.

پاسخ: $abba. کافی است چرخش‌های رشتهٔ $baba را مرتب می‌کنیم و ستون آخر را درنظر بگیریم.

$baba

a$bab

aba$b

ba$ba

baba$

ت. **[۴ نمره]** این جمله درست است یا غلط؟ چرا؟

روش وارسی درجهٔ دو (با استفاده از رابطهٔ $h(x,i) = (h'(x) + a \cdot i^{۲} + b \cdot i) \mod m$ با فرض ثابت بودن a و b) در درهم‌سازی با آدرس‌دهی باز، شرط درهم‌سازی یکنواخت را برآورده می‌کند؛ یعنی دنبالهٔ وارسی کلید k به احتمال یکسان، برابر با یکی از $m!$ جایگشت اعداد ۰ تا $m - ۱$ خواهد بود.

پاسخ: غلط. تمام کلیدهایی که $h'(k)$ یکسان دارند، جایگشت یکسانی خواهند داشت. دلایل دیگر نیز درصورت درست بودن قابل قبول است.

ث. **[۴ نمره]** به اختصار توضیح دهید با استفاده از آرایهٔ پسوندی و آرایهٔ کمکی طولانی‌ترین پیشوند مشترک (LCP) برای یک رشتهٔ s، چطور می‌توان کوچکترین زیررشتهٔ ناتهی s را پیدا کرد که فقط یک بار در s تکرار شده است؟

پاسخ: تمام خانه‌های آرایهٔ پسوندی را به صورت زیر بررسی می‌کنیم. اگر SA آرایه پسوندی باشد و $SA[i] = j$ باشد، در این صورت کوچکترین رشتهٔ یکتای s که از $s[j]$ شروع می‌شود را می‌توان با کمک LCP و مقایسه با $s[SA[i - ۱]]$ و $s[SA[i + ۱]]$ در $O(۱)$ پیدا کرد. در نهایت کمترین مقدار پیدا شده را خروجی می‌دهیم.

ج. **[۴ نمره]** یک داده‌ساختار پشته داریم که علاوه بر pop() (با هزینهٔ c) و push(x) (با هزینهٔ c)، دارای عمل multipop(k) (با هزینهٔ kc) باشد که معادل k بار اجرای pop() است. می‌خواهیم با تحلیل تابع پتانسیل ثابت کنیم هزینهٔ سرشکن هر عمل $O(۱)$ است. یک تابع پتانسیل مناسب برای این کار پیشنهاد دهید. چرا این تابع خوب کار می‌کند؟

پاسخ: تعداد اعضای پشته ضرب در c. در صورتی که ضرب در c لحاظ نشود، نیمی از نمره داده می‌شود.

چ. **[۴ نمره]** یک گراف وزن‌دار داریم که وزن یال‌ها می‌تواند یک عدد صحیح مثبت، ۰، -۱ و یا -۲ باشد. آیا می‌توان به وزن همهٔ یال‌ها ۳ واحد اضافه کرد و با اجرای الگوریتم دکسترا کوتاهترین مسیرها از رأس مشخص s به بقیه را پیدا کرد؟ اگر بله به اختصار توضیح دهید و اگر خیر یک مثال نقض ارائه دهید.

پاسخ: خیر. گراف سه رأسی مسیر $P_۳$ و رأس t را به عنوان یک مقصد در نظر بگیرید. اگر برای وزن یال‌ها داشته باشیم $w(s,t) = -۱$، $w(s,v) = ۰$ و $w(v,t) = -۲$، در این صورت مسیر به درستی پیدا نمی‌شود.

ح. **[۵ نمره]** یک درخت ریشه‌دار T داریم و می‌خواهیم از روی آن داده‌ساختاری بسازیم که هر گاه دو رأس از T به ما داده شد، بتوانیم پایین‌ترین جد مشترکشان را بیابیم. برای این کار روی درخت T، جست‌وجوی عمق اول (DFS) را اجرا می‌کنیم و هر موقع به یک رأس وارد می‌شویم (چه برای بار اول و چه وقتی از بچه‌های آن رأس برمی‌گردیم)، عمق رأس را در یک لیست یادداشت می‌کنیم. فرض کنید برای هر رأس اولین و آخرین اندیس مربوط به آن در لیست را داریم و برای هر خانهٔ لیست هم می‌دانیم مربوط به کدام

رأس است. با کمک این لیست چطور می‌توانیم به پرسش پایین‌ترین جد مشترک دو رأس پاسخ دهیم؟ از بین داده‌ساختارهایی که در این درس شنیده‌اید کدام را پیشنهاد می‌کنید؟ زمان ساخت داده ساختار و پاسخ به پرسش آن را بیان کنید.

پاسخ: برای پاسخ به هر پرسش برای رأس a و b (با فرض این که اولین خانه مربوط به a در لیست قبل از اولین خانه مربوط به b در لیست است)، کافی است در زیربازهٔ اولین اندیس مربوط به a تا آخرین اندیس مربوط به b کمترین عمق را پیدا کنیم که معادل «مسئلهٔ پرسش کمینهٔ یک محدوده» است که در جلسهٔ ۱۷ مطرح شد و بهترین راه آن (که فقط نام برده شد)، در $O(n)$ ساخته می‌شود و پاسخ به هر پرسش در $O(1)$ انجام می‌شود. توجه کنید طول لیست با وجود این که هر رأس چند بار در آن ظاهر شده است با فرض $n > 1$، $2e - 2 = 2n - 4 = O(n)$ هست و یعنی طول لیست با تعداد رأس‌ها از لحاظ مجانبی برابر است.

سؤال ۲. [۴۸ نمره] پیشنهاد الگوریتم یا داده‌ساختار مناسب. در هر قسمت از این سؤال از شما خواسته می‌شود یک الگوریتم یا داده‌ساختار مناسب پیشنهاد دهید. این داده‌ساختار یا الگوریتم می‌تواند از مطالب گفته شده در درس، ترکیبی از آن‌ها و یا خلاقیتی باشد. علاوه بر کلیّت داده‌ساختار و الگوریتم، زمان اجرا و دلیل انتخاب خود را نیز در ۴ الی ۵ خط به طور واضح توضیح دهید. همچنین در صورتی که زمان اجرای الگوریتم شما سرشکن و یا تصادفی است، آن را بیان کنید. در این سؤال نیاز به اثبات درستی الگوریتم و اثبات زمان اجرای آن نیست.

ورودی برخی سؤال‌ها یک گراف است، اما برای حل مسئله ممکن است لازم باشد گراف جدیدی بسازید. در این موارد روش ساخت گراف را توضیح دهید.

آ. **[۸ نمره]** در این مسئله، کلمه دنباله‌ای حداکثر ۱۰ حرفی از حروف الفبای فارسی تعریف می‌شود. همچنین جمله دنباله‌ای از کلمه‌ها تعریف می‌شود. دو جملهٔ $A = (a_1, \ldots, a_k)$ و $B = (b_1, \ldots, b_k)$ یکریخت هستند اگر الگوی تکرار کلمات در آن‌ها یکسان باشد، یعنی برای هر i و j، $a_i = a_j$ باشد اگر و تنها اگر $b_i = b_j$. در مثال زیر، جمله‌های آ و پ یکریخت هستند زیرا کلمات ۵اُم و ۷اُم در هر کدام از آن‌ها یکسان هستند و بقیه کلماتشان متمایز هستند؛ ولی جمله‌های آ و ب یکریخت نیستند.

آ = (”رنج“، ”گرانم“، ”را“، ”به“، ”صحرا“، ”می‌دهم“، ”صحرا“، ”نمی‌گیرد“)

ب = (”رمز“، ”میستان“، ”همه“، ”تو“، ”راز“، ”نیستان“، ”همه“، ”تو“)

پ = (”اشک“، ”روانم“، ”را“، ”به“، ”دریا“، ”می‌دهم“، ”دریا“، ”نمی‌گیرد“)

لیستی از n جمله به شما داده شده که هر جمله دقیقاً از k کلمه تشکیل شده است. یک الگوریتم با زمان $O(nk)$ طراحی کنید که جمله‌ها را به دسته‌های یکریخت افراز کند.

پاسخ: ایده: مرحلهٔ اول تبدیل هر جمله به یک دنباله از اعداد به صورتی که جمله‌های یکریخت به

دنباله‌های یکسان تبدیل شوند و جمله‌های غیریکریخت به دنبال‌های متفاوت. مرحلهٔ دوم دسته‌بندی جمله‌ها بر اساس دنبالهٔ متناظر به دست آمده در مرحلهٔ اول.

ابتدا نگاشتی از یک جمله به یک لیست از اعداد پیدا می‌کنیم که دو جمله یکریخت هستند اگر و تنها اگر لیست مربوط به آن‌ها یکی باشد. برای این کار کلمات جمله را به ترتیب در یک جدول درهم‌سازی (یا ترای) جست‌وجو می‌کنیم. اگر وجود داشت یعنی این کلمه تکراری است و عدد مربوط به آن را در لیست قرار می‌دهیم. در غیر این صورت این کلمه جدید است و آن را در جدول (یا ترای) وارد کرده و عدد جدیدی به آن اختصاص می‌دهیم. اعدادی که اختصاص می‌دهیم می‌توانند اعداد طبیعی با شروع از یک باشند. در این صورت نگاشت یک جمله به یک لیست $O(k)$ زمان می‌گیرد که در صورت استفاده از جدول درهم‌سازی زمان تصادفی است و در صورت استفاده از ترای زمان بدترین حالت است. بنابراین نگاشت همهٔ جمله‌ها حداکثر $O(nk)$ زمان می‌برد. حال کافی است جمله‌ها را توجه به لیست‌شان به دسته‌های هم‌ارزی افراز کنیم. برای این کار می‌توان از یک جدول درهم‌سازی جدید استفاده کرد و وقتی لیست هر جمله را در آن جدول وارد می‌کنیم، اگر تا کنون در آن جدول وجود نداشت یک آرایهٔ پویا برای جمله‌های یکریخت با آن با درج اندیس جمله در ابتدای آرایه می‌سازیم و اگر در جدول قبلاً درج شده بود، آرایهٔ مربوط به جمله‌های یکریخت با آن را پیدا کردن و اندیس جمله را به انتهای آن آرایه اضافه می‌کنیم.

- درهم‌سازی اول را می‌توان با ترای جایگزین کرد و درهم‌سازی دوم را با مرتب‌سازی مبنایی.
- توجه کنید نیاز به توضیح چگونگی کارکرد جدول درهم‌سازی نیست (مگر این که بخواهیم تغییری روی آن ایجاد کنیم).
- همچنین توجه کنید که پیدا کردن دسته‌های هم‌ارزی با زنجیر جدول درهم‌سازی (بدون ایدهٔ جدید) ممکن نیست، زیرا اگر یک کلید را دوبار در جدول وارد کنیم، اولی پاک می‌شود.
- زمان درهم‌سازی تصادفی است و ذکر این نکته ضروری است.

ب. **[۸ نمره]** در راستای یک خیابان شرقی‌ـ‌غربی، برج‌های بلندی در حال ساخت و خراب شدن هستند. یک برج در صورتی می‌تواند طلوع خورشید را ببیند که ارتفاع آن از ارتفاع برج‌های سمت شرقش اکیداً بیشتر باشد. داده‌ساختاری برای نگه‌داری برج‌های این خیابان طراحی کنید که اعمال زیر را در زمان $O(\log n)$ پشتیبانی کند (n تعداد برج‌ها در هر لحظه است).

- `add(adr, hei)`: یک برج جدید با ارتفاع h که یک عدد صحیح مثبت است، در آدرس adr که آن نیز یک عدد صحیح مثبت است، اضافه شود.
- `remove(adr)`: برجی که در آدرس adr هست از داده‌ساختار حذف شود.
- `sunrise(adr)`: آیا برجی که در آدرس adr هست طلوع خورشید را می‌بیند؟ اگر بله مقدار ۱ وگرنه مقدار ۰ برگردانده شود.

پاسخ: برای این کار از یک درخت AVL (یا قرمز_سیاه) افزوده استفاده می‌کنیم که هر رأس بیشینهٔ زیردرخت خود را دارد. در این داده‌ساختار همان‌طور که در کلاس گفته شده، می‌توان یک عنصر را در $O(\log n)$ درج یا حذف کرد و یا بیشینهٔ یک زیربازه را به دست آورد. توضیح کارکرد و درستی داده‌ساختار در جواب سؤال نیاز نیست.

کافی است برج‌ها را بر حسب adr در درخت درج کنیم و بر اساس ارتفاع برج‌ها اطلاعات افزوده را نگه داریم. در این صورت دو عمل اول به راحتی انجام می‌شود و برای عمل سوم کافی است بیشینه ارتفاع برج‌های قبل از adr را با این داده‌ساختار به دست آوریم و آن را با ارتفاع برجی که در adr قرار دارد مقایسه کنیم. در این صورت هر سه عمل در $O(\log n)$ انجام می‌شوند.

اگر به جای نگه‌داری بیشینهٔ زیردرخت در هر رأس، مثلاً بخواهیم بیشترین عنصر در سمت شرق را نگه داریم در موقع به‌روزرسانی به مشکل خواهیم خورد. همچنین ممکن است یکی به اشتباه فرض کنید که رأس‌های سمت شرق در زیردرخت یک رأس وجود دارند که راه‌حل را غلط می‌کند.

زمان درخت پاره‌خطی به حداکثر مقدار adr وابسته است و بنابراین ۶ نمره به آن تعلق می‌گیرد. درخت فنویک نیز عمل بیشینه را پشتیبانی نمی‌کند.

پ. **[۸ نمره]** آقای ووپی در جایی زندگی می‌کند که خیابان‌ها و کوچه‌ها شیب زیادی دارند. او می‌خواهد صبح از خانه تا محل کار را پیاده‌روی کند. محل زندگی او به صورت n تقاطع و e راه یک‌طرفه بین تقاطع‌ها مدل شده است که هر تقاطع یک ارتفاع منحصربه‌فرد دارد. خانه و محل‌کار آقای ووپی هم هر کدام یک تقاطع هستند. مسیر آقای ووپی از خانه به محل کار باید به صورتی باشد که ابتدا ارتفاع مسیر طی شده صعودی باشد، سپس در یک تقاطع کمی استراحت کند و آب بنوشد و سپس ارتفاع ادامهٔ مسیر طی شده نزولی باشد. در بین این مسیرها آقای ووپی می‌خواهد کوتاه‌ترین مسیر را پیدا کند. برای این کار یک الگوریتم کارا پیشنهاد دهید.

پاسخ: ایده: تبدیل گراف به یک DAG بر اساس ارتفاع‌ها و پیدا کردن کوتاهترین مسیر بر اساس وزن یال‌ها با الگوریتم مربوط به DAG.

گراف G را به این صورت می‌سازیم که برای هر تقاطع x دو رأس x_1 و x_2 قرار می‌دهیم. برای هر راه بین دو تقاطع x و y با وزن w، اگر ارتفاع x کمتر باشد، یال جهت‌داری از x_1 به y_1 با وزن w و اگر ارتفاع y کمتر باشد، یال جهت‌داری از x_2 به y_2 با وزن w قرار می‌دهیم. همچنین برای هر تقاطع x یال جهت‌داری با وزن صفر از x_1 به x_2 قرار می‌دهیم. در این گراف رأس‌های با اندیس ۱ مربوط به مسیر صعودی (قبل از توقف) است، یال‌های بین اندیس ۱ و ۲ برای توقف و آب خوردن است و در نهایت رأس‌های با اندیس ۲ مربوط به مسیر نزولی (بعد از توقف) است. توجه کنید گراف جدید بدون دور است (گراف اصلی می‌تواند دور داشته باشد). در این گراف با الگوریتم مربوط به DAG، کوتاهترین مسیر را

پیدا می‌کنیم. زمان اجرای الگوریتم $O(n+e)$ است.

اگر به جای الگوریتم کوتاه‌ترین مسیر در DAG، از دو بار اجرای دکسترا (یک بار از خانه و یک بار برعکس از محل کار) استفاده شود، ۶ نمره داده خواهد شد. همچنین اگر گراف مشابهی ساخته شود ولی در نهایت از دکسترا استفاده شود. همچنین اگر دو دکسترا زده شود ولی وقتی برای اولین بار به هم رسیدند متوقف شوند، ۵ نمره داده خواهد شد.

در صورتی که از الگوریتم بلمن‌فورد به صورت درستی استفاده شود یا دکسترا از n رأس دکسترا زده شود ۳ نمره داده خواهد شد.

ت. **[۸ نمره]** رئیس آقای ووپی، آقای استنلی برای پیاده‌روی از منزل به محل کار به دنبال راه ایده‌آل از نظر خودش است. مدل تقاطع‌ها و راه‌ها مثل قسمت قبل است (بدون درنظر گرفتن ارتفاع‌ها). در هر کدام از e راه گفته شده در سؤال قبل، حداقل یک سگ ولگرد زندگی می‌کند. آقای استنلی از سگ‌ها می‌ترسد و برای همین به دنبال راهی هست که با کمترین سگ در مسیر خود روبه‌رو شود. اگر تعداد سگ‌ها k باشد، الگوریتمی با زمان $O(k)$ برای این کار طراحی کنید.

پاسخ: ایده: جست‌وجوی سطح اول (BFS) روی گرافی که راه‌ها در صورت وجود چند سگ، به چند یال تبدیل شده‌اند.

گراف G را به این صورت می‌سازیم که برای هر تقاطع یک رأس می‌گذاریم و برای هر راه، دو تقاطع دو سر راه را با یک مسیر x یالی به هم وصل می‌کنیم که x تعداد سگ‌های موجود در آن راه است. در این صورت تعداد رأس‌های گراف G برابر با $n+(k-e)$ و تعداد یال‌های این گراف برابر با k خواهد بود. با توجه به این‌که هر راه حداقل یک سگ دارد و به هر تقاطع حداقل یک راه وصل است (وگرنه اسمش تقاطع نبود!)، می‌توان گفت تعداد رأس‌ها و یال‌ها $O(k)$ است و گراف بدون وزن است. حال از جست‌وجوی سطح اول (BFS) استفاده می‌کنیم که مسیر با کمترین یال را از خانه به محل کار آقای استنلی در زمان $O(k)$ پیدا کنیم.

توجه کنید گراف برخلاف مسئلهٔ قبل دیگر DAG نیست. همچنین استفاده از دکسترا کندتر از BFS خواهد بود (مگر این که به جای فیبوناچی هیپ، از این نکته استفاده کنیم که dها یک عدد صحیح بین صفر تا حداکثر k هستند و از یک جدول دسترسی مستقیم به عنوان صف اولویت استفاده کنیم که تحلیل آسانی هم ندارد و این راه توسط کسی نوشته نشده است).

ث. **[۸ نمره]** آقای ووپی می‌خواهد از محل کار خود به یک جلسهٔ مهم برود. با توجه به این که ساعت اوج ترافیک است، او می‌خواهد خود را با ماشین به یک ایستگاه مترو برساند و با مترو به محل جلسه برود. مدل تقاطع‌ها و راه‌ها مثل دو قسمت قبل است و m ایستگاه مترو نیز در تقاطع‌های مربوط به خود وجود دارند. هدف این است که آقای ووپی را در سریع‌ترین زمان ممکن به یک ایستگاه مترو برسانیم. الگوریتم

بهینه‌ای برای این کار پیشنهاد دهید.

پاسخ: تفاوت این مسئله با مسئله‌ای که یک مبدأ و یک مقصد داریم در وجود چندین مقصد است. برای این کار می‌توان همهٔ مقصدها را به یک رأس جدید با یال به طول صفر وصل کرد و آن را مقصد جدید گرفت و یا مسئلهٔ کوتاهترین مسیرها از یک مبدأ را برای همهٔ رأس‌های دیگر حل کرد و در نهایت بین همهٔ رئوس مربوط به ایستگاه‌های مترو کمترین مقدار را انتخاب کرد. الگوریتم استفاده شده دکسترا و زمان آن $O(n \log n + e)$ خواهد بود.

ج. **[۸ نمره]** یک گراف جهت‌دار G داریم که یال‌های آن وزن مثبت یا منفی دارند. الگوریتمی با زمان $O(ne)$ پیشنهاد دهید که مشخص کند آیا مسیری از s به t وجود دارد که وزن آن حداقل w باشد یا خیر؟

پاسخ: وزن یال‌ها را منفی کرده و از الگوریتم بلمن‌فورد استفاده می‌کنیم. در صورتی که جواب بلی باشد، یا طولانی‌ترین مسیر از s به t یک مسیر ساده است و وزنش بیشتر از w است و یک مسیر غیرساده از s به t وجود دارد که از یک دور مثبت (در گراف اصلی) استفاده کرده است.

به صورت دقیق‌تر، پس از منفی کردن وزن یال‌ها، الگوریتم بلمن‌فورد را اجرا کرده و وجود دور منفی را بررسی می‌کنیم. اگر دور منفی بررسی می‌کنیم که آیا وزن کوتاه‌ترین مسیر از s به t کمتر یا مساوی $-w$ است یا خیر. همچنین همان‌طور که در کلاس گفته شد، n دور دیگر الگوریتم بلمن‌فورد را اجرا می‌کنیم و می‌بینیم که آیا طول مسیر تغییر کرد یا نه. اگر تغییر کرد به معنی وجود مسیر غیرساده از از s به t با عبور از یک دور منفی در این گراف (معادل یک دور مثبت در گراف اصلی) است و جواب بله است. اگر هم تغییر نکرد به معنی عدم وجود این مسیر و جواب منفی است. زمان اجرای الگوریتم $O(ne)$ است.

سؤال ۳. [۱۹ نمره] رأس‌های برشی. یک رأس در گراف بدون جهت G برشی است اگر و تنها اگر حذف آن از گراف، تعداد مؤلفه‌های همبندی گراف را بیشتر کند. ما در این مسئله می‌خواهیم رئوس برشی گراف همبند و بدون جهت G را پیدا کنیم.

در این قسمت باید اثبات دقیقی برای درستی الگوریتم و زمان اجرای آن بیان کنید.

آ. **[۱۰ نمره]** فرض کنید جست‌وجوی عمق اول (DFS) را از یک رأس دلخواه اجرا می‌کنیم. ثابت کنید رأس غیربرگ و غیرریشه v از درخت DFS برشی است اگر و تنها اگر زیردرخت یکی از فرزندان v یال برگشتی به اجداد v نداشته باشد.

پاسخ: با توجه به این که ادعا از نوع «اگر و تنها اگر» است، آن را باید از دو جهت اثبات کنیم.

جهت اول: اگر u یک رأس برشی باشد، زیردرخت یکی از فرزندان آن هیچ یال برگشتی به اجداد u ندارد. این ادعا را می‌توان با برهان خلف اثبات کرد. فرض کنید تمام زیردرخت فرزندان یک یال بازگشتی به اجداد u داشته باشند. اگر رأس u از گراف حذف شود، تمام زیردرخت‌های u با یال بازگشتی مذکور به

جدی از u متصل هستند و بنابراین گراف همبند می‌ماند که نتیجه می‌دهد u برشی نیست. این تناقض با فرض برشی بودن u نتیجه می‌دهد فرض خلف غلط است و حداقل یکی از زیردرخت‌های u یال بازگشتی به اجداد u ندارد.

جهت دوم: اگر زیردرخت یکی از فرزندان u هیچ یال برگشتی به اجداد u نداشته باشد، u یک رأس برشی است. برای نشان‌دادن درستی این ادعا نیز کافی است توجه کنیم در DFS روی یک گراف بدون‌جهت، تنها یال‌های درخت DFS وجود دارند و یال‌های بازگشتی (و یال‌های متقاطع و رو به جلو وجود ندارند). بنابراین هیچ مسیری از زیردرخت فرزندان u به یکدیگر و به رأس‌هایی که اجداد و نوادگان u نیستند وجود ندارد. با توجه به این فرض که یال‌های بازگشتی نیز در این مسئله وجود ندارند، تنها یال‌های درخت بین زیردرخت موردنظر و خارج از آن وجود دارد. بنابراین اگر u از گراف حذف شود، زیردرخت موردنظر از بقیهٔ رئوس جدا شده و گراف غیرهمبند می‌شود. در نتیجه u برشی است.

با اثبات جهت اول و دوم، ادعای خواسته شده ثابت می‌شود.

در صورتی که با یال‌های متقاطع اشاره نشود، حداقل ۲ نمره کم می‌شود.

ب. **[۹ نمره]** از این ایده استفاده کنید و الگوریتمی با زمان اجرای $O(n+e)$ برای پیدا کردن همهٔ رأس‌های برشی یک گراف ارائه داده و اثبات کنید.

پاسخ: شرط لازم و کافی برای برشی بودن رأس‌های میانی (غیر از برگ‌ها و ریشه) در قسمت آ گفته شده است. برگ‌ها برشی نیستند زیرا با حذف آن‌ها درخت DFS بقیه رئوس را به هم متصل می‌کند. ریشه نیز برشی اگر و تنها اگر بیش از یک فرزند در درخت DFS داشته باشد. برای اثبات این ادعا توجه کنید اگر ریشه تنها یک فرزند داشته باشد مشابه برگ‌ها با استفاده از درخت DFS برشی نمی‌تواند باشد و اگر بیش از یک فرزند داشته باشد، چون یال متقاطع وجود ندارد، ریشه حتماً برشی است.

برای بررسی این شروط کافی است از یک رأس دلخواه جست‌وجوی عمق اول را اجرا کنیم و برای هر رأس، عمق خود رأس (یا به جای آن زمان شروع) را نگه داریم. با استفاده از این عمق، برای هر رأس، کم‌عمق‌ترین همسایهٔ آن را نگه داشته و با استفاده از این، برای هر رأس کم‌عمق‌ترین همسایهٔ زیردرخت رأس را می‌توانیم محاسبه کنیم. با داشتن این اطلاعات می‌توان شرط قسمت آ را به راحتی در حین اجرای DFS بررسی کرد. شرط رأس‌های برگ و ریشه نیز در حین اجرای DFS قابل انجام است. زمان محاسبه اطلاعات اضافه و چک کردن شرط‌ها برای رأس v حداکثر به اندازه $O(\deg_G(v))$ (متناسب با درجهٔ v) است و از لحاظ مجانبی، زمان بیشتری را به DFS اضافه نمی‌کند. بنابراین زمان اجرای الگوریتم $O(n+e)$ خواهد بود.